



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto DataQuest

Curso: Base de Datos II

Docente: Patrick Jose Cuadros Quiroga

Integrante:

Dongo Palza,Manuel Andree

(2023076842)

Perez Peralta,Fabrizio Salvador

(2023077476)

Tacna - Perú

2026

Proyecto DataQuest: Validador de Normalización de Bases de Datos Relacionales

Versión: 1.0

Documento de Estándares de Programación

Versión 1.0

Este documento define las reglas técnicas mínimas que debe cumplir el código fuente, la API, la interfaz Streamlit, el acceso a datos Supabase/PostgreSQL y las pruebas del sistema DataQuest. Su finalidad es garantizar una implementación clara, mantenible, segura, trazable y coherente con el objetivo académico del proyecto.

Alcance del estándar

- Aplica a módulos Python del motor de normalización, parser SQL, diagnóstico, corrector y generadores de reportes.
- Aplica a la API FastAPI, modelos Pydantic, endpoints y manejo uniforme de errores.
- Aplica a la interfaz Streamlit y su interacción con controladores, repositorios y servicios.
- Aplica a la persistencia en Supabase/PostgreSQL y al uso del esquema real: usuarios, perfiles, presencia e historial_validaciones.
- Aplica a pruebas unitarias, de integración, UI, seguridad básica y control de versiones.

Historia de Revisión

Ítem	Fecha	Versión	Descripción	Equipo / responsable
1	2026	1.0	Versión inicial del documento de estándares de programación para DataQuest.	Manuel Dongo Fabrizio Perez

Contenido

Historia de Revisión	3
Estándares de Programación	4
1. OBJETIVO	4
2. DECLARACIÓN DE VARIABLES	4
2.1 Descripción de variables	5
2.2 Variables de identificación	5
2.3 Variables de datos principales	6
2.4 Variables de control de flujo y trazabilidad	6
2.5 Variables de API	6
2.6 Variables de interfaz Streamlit	7
3. DEFINICIÓN DE DATOS Y FUNCIONES.....	7
3.1 Tipos de datos	7
3.2 Declaración de variables.....	7
3.3 Declaración de funciones.....	8
3.4 Manejo de errores.....	8
3.5 Validaciones con Pydantic	9
3.6 Control de versiones de código fuente	9
4. PROCEDIMIENTOS Y FUNCIONES DEFINIDOS POR EL USUARIO	9
4.1 Métodos equivalentes a procedimientos almacenados.....	9
4.2 Funciones puras	10
4.3 Funciones de tabla o procesamiento de colecciones.....	10
5. ESTÁNDARES DE BASE DE DATOS.....	11
5.1 Tablas reales consideradas por el estándar	11
6. ESTÁNDARES DE API FASTAPI	11
7. ESTÁNDARES DE INTERFAZ STREAMLIT	12
8. ESTÁNDARES DE SEGURIDAD	12
9. ESTÁNDARES DE PRUEBAS Y QA.....	13
9.1 Criterios de aceptación QA	13
10. BENEFICIOS	13
11. CONCLUSIONES.....	14
12. RECOMENDACIONES	14

Estándares de Programación

1. OBJETIVO

El objetivo del presente documento es establecer un conjunto de reglas obligatorias de programación para DataQuest, con el propósito de asegurar que el código fuente, las estructuras de datos, los servicios, la API, la interfaz web y las pruebas se desarrollen de manera ordenada, segura, mantenible y trazable. DataQuest es un sistema académico que analiza esquemas de bases de datos relacionales, diagnostica el cumplimiento de 1FN, 2FN y 3FN, registra historial de validaciones y presenta sugerencias técnicas de mejora.

Este estándar busca reducir ambigüedades, evitar duplicidad de código, facilitar la revisión del docente, mejorar la calidad de las entregas y permitir que futuros desarrolladores comprendan rápidamente la arquitectura del sistema. Su aplicación será obligatoria en nuevos módulos y recomendada en la refactorización de componentes existentes.

- Asegurar la validez de los datos de entrada mediante modelos Pydantic, validaciones explícitas en Python y restricciones CHECK en PostgreSQL/Supabase.
- Centralizar el flujo de análisis en componentes separados: parser, validadores, diagnóstico, sugerencias, persistencia y presentación.
- Registrar fechas de auditoría como creado_en, conectado_en, ultima_actividad y fecha para mejorar trazabilidad y depuración.
- Evitar que la interfaz Streamlit contenga lógica pesada de normalización; la interfaz debe invocar funciones de servicio o controladores.
- Estructurar el proyecto por capas: views, controllers, core, db, tests, api.py y documentación.
- Asegurar que DataQuest analice SQL como texto, sin ejecutar sentencias del usuario contra bases reales.
- Promover pruebas unitarias, de integración y de interfaz que respalden la confiabilidad del motor de normalización.

Capas esperadas de DataQuest:

Capa	Responsabilidad	Ejemplos reales / sugeridos
Interfaz Streamlit	Captura entradas, muestra resultados, historial y mensajes amigables.	views/04_validador.py, views/05_resultados.py, views/06_historial.py
API FastAPI	Expone servicios HTTP/JSON para consumo externo.	api.py, endpoint POST /api/normalizacion
Controladores	Orquestan interacción entre UI, motor y base de datos.	validacion_controller.py, auth_controller.py, dashboard_controller.py
Motor core	Parsea SQL, valida normalización, diagnostica y genera sugerencias.	parser.py, validador_1fn.py, validador_2fn.py, validador_3fn.py, diagnostico.py, corrector.py
Persistencia	Gestiona conexión y operaciones sobre Supabase/PostgreSQL.	db/conexion.py, db/auth.py, db/realtime.py, db/modelos.sql
Pruebas	Verifica comportamiento funcional, integración y UI.	tests/unit, tests/integration, tests/ui, tests/bdd

2. DECLARACIÓN DE VARIABLES

DataQuest debe aplicar el estándar PEP 8 de Python para declaración de variables, funciones, clases y módulos. Los nombres deben ser descriptivos, consistentes y relacionados con el dominio de normalización de bases de datos. Se debe evitar la mezcla arbitraria de idiomas y se debe mantener una nomenclatura técnica uniforme.

- Variables y funciones: snake_case.
- Constantes: MAYÚSCULAS con guion bajo.
- Clases: PascalCase.
- Métodos privados: guion bajo inicial.
- Tablas y columnas SQL: snake_case.
- Evitar nombres crípticos como x, tmp, dato1, arreglo, valor o prueba cuando no aporten contexto.

- Los nombres deben indicar intención: nivel_objetivo, esquema_original, violaciones, sugerencias, historial_id.

Alcance	Convención	Ejemplo DataQuest	Descripción
Global / constante	MAYÚSCULAS	NIVELES_VALIDOS, FORMATOS_ENTRADA, MAX_SQL_LENGTH	Valores fijos usados por múltiples módulos.
Local	snake_case	nombre_esquema, formato_entrada, nivel_objetivo	Variables usadas dentro de funciones o bloques específicos.
Privado	_snake_case	_parse_create_table, _build_response	Funciones auxiliares internas que no deben exponerse como API pública.
Clase	PascalCase	NormalizacionService, SQLParser, SupabaseRepository	Componentes que encapsulan responsabilidad y comportamiento.
Función	verbo_sustantivo	analizar_esquema, validar_1fn, guardar_historial	Acciones concretas ejecutadas sobre el dominio del sistema.

2.1 Descripción de variables

Cada variable debe declarar o evidenciar su propósito dentro del flujo de DataQuest. En funciones críticas se recomienda usar anotaciones de tipo para mejorar lectura, validación estática y mantenimiento. Las variables que representen objetos JSON deben tener nombres que indiquen claramente si almacenan entrada, resultado, violaciones o sugerencias.

Estructura	Regla estándar	Ejemplo correcto	Ejemplo incorrecto
Longitud	Sin límite rígido; se prioriza claridad.	dependencias_funcionales	dep
Formato	snake_case para variables y funciones.	esquema_original	EsquemaOriginal
Contexto	El nombre debe expresar dominio.	nivel_objetivo	nivel
Seguridad	Nunca exponer secretos en nombres o logs.	password_hash	password_plano
Consistencia	Mantener idioma técnico uniforme.	guardar_historial_validacion	save_historial_validacion

2.2 Variables de identificación

Variable	Tipo de dato	Descripción	Módulo
id	uuid / str	Identificador principal de tablas y objetos internos. En PostgreSQL se genera con gen_random_uuid().	Todos
user_id	uuid / str	Clave foránea que referencia a usuarios.id; permite asociar validaciones e información de presencia.	Historial / Presencia
username	text / str	Identificador único del usuario. Debe ser único y no debe contener datos sensibles innecesarios.	Autenticación
password_hash	text / str	Contraseña cifrada o hasheada; nunca debe almacenarse como texto plano.	Usuarios
historial_id	uuid / str	Identificador de una validación guardada en historial_validaciones.	Historial
perfil_id	uuid / str	Identificador del perfil asociado al usuario; coincide con usuarios.id en relación 1 a 1.	Perfiles

2.3 Variables de datos principales

Variable	Tipo de dato	Descripción	Módulo
nombre_esquema	str / text	Nombre descriptivo del esquema analizado por el usuario.	Validación
formato_entrada	str / text + CHECK	Formato recibido: sql, csv, xls, txt o texto_pegado.	Entrada / BD
nivel_inicial	str / text + CHECK	Nivel declarado o detectado antes de aplicar validación.	Normalización
nivel_objetivo	str / text + CHECK	Nivel al que se desea llevar el esquema: 1fn, 2fn o 3fn.	Normalización
nivel_final	str / text + CHECK	Nivel alcanzado tras el diagnóstico del motor.	Normalización
esquema_original	dict / jsonb	Representación estructurada del esquema ingresado.	Historial
esquema_final	dict / jsonb	Representación estructurada de la propuesta o corrección final.	Historial
dependencias	list[dict] / jsonb	Dependencias funcionales detectadas o ingresadas por el usuario.	Motor core
violaciones	list[dict] / jsonb	Lista de incumplimientos detectados por nivel de normalización.	Diagnóstico
sugerencias	list[dict] / jsonb	Recomendaciones generadas por el corrector.	Corrector

2.4 Variables de control de flujo y trazabilidad

Variable	Tipo de dato	Descripción	Módulo
fecha	datetime / timestampz	Fecha de ejecución de la validación.	Historial
creado_en	datetime / timestampz	Fecha de creación del usuario.	Usuarios
conectado_en	datetime / timestampz	Fecha en la que el usuario se conectó o inició actividad.	Presencia
ultima_actividad	datetime / timestampz	Marca de la última acción del usuario en comunidad o sesión.	Presencia
errores_detectados	int	Cantidad resumida de errores o violaciones encontradas.	Diagnóstico
cumple_1fn	bool	Indica si una tabla o esquema cumple Primera Forma Normal.	Validador 1FN
cumple_2fn	bool	Indica si una tabla o esquema cumple Segunda Forma Normal.	Validador 2FN
cumple_3fn	bool	Indica si una tabla o esquema cumple Tercera Forma Normal.	Validador 3FN
mensaje_error	str	Mensaje controlado para informar fallos sin exponer detalles internos.	Manejo de errores

2.5 Variables de API

Variable	Tipo de dato	Descripción	Regla
request_body	dict / Pydantic model	Cuerpo recibido por un endpoint de FastAPI.	Debe validarse antes de procesar.
response_payload	dict	Respuesta JSON estructurada enviada al cliente.	Debe mantener formato uniforme.
status_code	int	Código HTTP asociado al resultado.	Usar 200, 400, 404, 422 o 500 según corresponda.
api_base_url	str	URL base de la API usada por clientes externos.	No debe incluir tokens.
endpoint_normalizacion	str	Ruta funcional principal /api/normalizacion.	Debe documentarse en OpenAPI.
sql	str	Campo recibido por el endpoint para analizar sentencias CREATE TABLE.	No debe ejecutarse; solo analizarse como texto.

2.6 Variables de interfaz Streamlit

Variable	Tipo de dato	Descripción	Regla
sql_input	str	SQL ingresado o pegado por el usuario en la interfaz.	Debe validarse antes de enviarse al motor.
uploaded_file	file-like	Archivo cargado si se permite entrada por csv, xls o txt.	Validar extensión y tamaño.
resultado_validacion	dict	Resultado devuelto por el motor o API.	Debe renderizarse con tablas y mensajes claros.
estado_sesion	dict	Información almacenada en st.session_state.	Usar solo para estado necesario.
historial_usuario	list[dict]	Validaciones previas consultadas para el usuario.	Debe paginarse o limitarse si crece.
mensaje_ui	str	Mensaje amigable mostrado al usuario.	No debe exponer stack traces.

3. DEFINICIÓN DE DATOS Y FUNCIONES

3.1 Tipos de datos

Los tipos de datos deben ser coherentes entre Python, FastAPI/Pydantic y PostgreSQL/Supabase. En DataQuest, los resultados de normalización pueden variar de tamaño y estructura; por ello, el uso de jsonb en historial_validaciones está permitido para esquemas, dependencias, violaciones y sugerencias.

Uso	Tipo Python	Tipo PostgreSQL/Supabase	Razón de elección	Ejemplos
Identificadores	UUID / str	uuid	Evita colisiones y permite relación entre tablas.	id, user_id
Texto	str	text	Flexible y sin longitud artificial.	username, nombre_esquema
Fechas	datetime	timestamp with time zone	Permite trazabilidad con zona horaria.	fecha, creado_en
Booleanos	bool	boolean	Estados de cumplimiento y banderas internas.	cumple_1fn, cumple_2fn
JSON	dict / list	jsonb	Permite guardar resultados dinámicos del motor.	esquema_original, violaciones
Enumerados	str	text + CHECK	Restringe valores permitidos desde la base de datos.	formato_entrada, nivel_final
Listas	list[dict]	jsonb	Almacena dependencias y sugerencias variables.	dependencias, sugerencias
Errores	str / dict	text / jsonb	Permite describir fallas controladas.	mensaje_error, error_payload

3.2 Declaración de variables

- Declarar variables cerca de su uso para reducir el alcance accidental.
- No reutilizar una variable con significados distintos dentro de la misma función.
- Inicializar colecciones con valores seguros, por ejemplo [] o {}, antes de llenarlas.
- Evitar variables globales mutables para no afectar sesiones concurrentes en Streamlit o FastAPI.
- Usar anotaciones de tipo en funciones críticas del motor, controladores y repositorios.
- Usar Optional solo cuando el valor realmente puede estar ausente.
- Validar entradas externas antes de procesarlas.
- No imprimir contraseñas, tokens, claves de Supabase ni datos sensibles en logs.

Ejemplos correctos:

```
nombre_esquema: str = "Sistema de Matrícula"
nivel_objetivo: str = "3fn"
violaciones: list[dict] = []
sugerencias: list[dict] = []
```

Ejemplos incorrectos:

```
x = "3fn"          # No expresa intención
data = []          # Demasiado genérico
passw = "123456"   # Prohibido almacenar contraseña plana
resultado = None    # Debe validarse antes de usarse
```

3.3 Declaración de funciones

Las funciones deben tener una responsabilidad clara. El parser no debe guardar datos, el validador no debe renderizar interfaz, y la interfaz no debe contener lógica profunda de normalización. La separación de responsabilidades permite probar el motor sin depender de Streamlit ni de Supabase.

- Usar nombres con verbo + sustantivo: parsear_esquema, validar_1fn, guardar_historial.
- Evitar funciones demasiado largas o con múltiples responsabilidades.
- Separar parsing, validación, diagnóstico, sugerencias y persistencia.
- Retornar estructuras consistentes: dict con claves esperadas, listas de violaciones o modelos Pydantic.
- Documentar parámetros y retorno con docstrings cuando la función sea parte del core.
- Manejar errores esperados y devolver mensajes controlados.
- No consultar Supabase directamente desde vistas si existe un controlador o repositorio.

Tipo de función	Convención	Ejemplo DataQuest	Propósito
Parser	verbo + objeto	parse_schema(input_data, formato, fds_input="")	Convertir entrada SQL, CSV, XLS o texto en estructura de esquema.
Parser SQL	verbo + objeto	parse_sql(sql_text)	Extraer tablas, columnas, PK y FK desde CREATE TABLE.
Dependencias	verbo + objeto	parse_fds(fds_input)	Interpretar dependencias funcionales ingresadas por el usuario.
Validación 1FN	validar_nivel	validar_1fn(tabla)	Detectar atributos multivaluados o no atómicos.
Validación 2FN	validar_nivel	validar_2fn(tabla, fds)	Detectar dependencias parciales.
Validación 3FN	validar_nivel	validar_3fn(tabla, fds)	Detectar dependencias transitivas.
Diagnóstico	verbo + objeto	diagnosticar_esquema(esquema_parseado)	Consolidar resultado de normalización.
Corrector	verbo + objeto	generar_sugerencias(diagnostico)	Proponer mejoras y correcciones.
Persistencia	verbo + objeto	guardar_historial(user_id, nombre_esquema, schema_parseado, diagnostico)	Guardar evidencia de validación.
Interfaz	verbo + objeto	mostrar_resultado_validacion(resultado)	Renderizar resultado sin modificar reglas de negocio.

3.4 Manejo de errores

El manejo de errores debe impedir que fallos internos se expongan al usuario final. En FastAPI se debe usar HTTPException o respuestas JSON controladas. En Streamlit se deben mostrar mensajes comprensibles. En el motor core se deben propagar excepciones con información útil para depuración, sin revelar credenciales ni rutas sensibles.

Tipo de error	Causa probable	Respuesta esperada	Módulo
Entrada vacía	El usuario no ingresó SQL ni archivo.	Mensaje: El campo SQL no puede estar vacío.	API / UI
SQL inválido	No se detectan sentencias CREATE TABLE válidas.	Mensaje de corrección de sintaxis, sin stack trace.	Parser
Formato no permitido	formato_entrada fuera de sql, csv, xls, txt o texto_pegado.	Error 400/422 y validación Pydantic.	API / BD
Nivel no permitido	nivel_objetivo distinto de 1fn, 2fn o 3fn.	Error controlado por CHECK o modelo Pydantic.	API / BD
Fallo Supabase	Error de conexión o credencial ausente.	Mensaje técnico controlado y log interno.	Persistencia
Dependencias mal escritas	Formato incorrecto en A,B -> C.	Advertencia y guía de formato esperado.	Motor core

3.5 Validaciones con Pydantic

Las entradas expuestas por API deben validarse con Pydantic antes de llegar al motor. El objetivo es rechazar datos incompletos o mal formados y garantizar que el motor core reciba estructuras predecibles.

```
class RequestNormalizacion(BaseModel):  
    sql: str = Field(..., description="Sentencias CREATE TABLE del esquema relacional.")  
  
    @field_validator("sql")  
    def validar_sql_no_vacio(cls, value: str) -> str:  
        if not value or not value.strip():  
            raise ValueError("El SQL no puede estar vacío")  
        return value.strip()
```

Reglas adicionales para modelos de API:

- Definir modelos de entrada y salida; no devolver respuestas improvisadas con estructuras diferentes.
- Usar Field para describir campos visibles en Swagger/OpenAPI.
- Aplicar validadores para formato_entrada, nivel_objetivo y tamaño máximo del SQL.
- No aceptar campos no necesarios si podrían generar ambigüedad o riesgo de seguridad.
- Mantener compatibilidad entre modelos Pydantic y restricciones CHECK de la base de datos.

3.6 Control de versiones de código fuente

Todo cambio relevante debe registrarse mediante Git. Los commits deben describir la intención del cambio y evitar mensajes genéricos. El repositorio no debe incluir credenciales, archivos .env, tokens, cachés, archivos temporales ni datos sensibles.

Regla	Descripción	Ejemplo
Ramas	Usar main para entregas estables, develop para integración y feature/nombre para cambios nuevos.	feature/validacion-2fn
Commits	Utilizar prefijos claros según tipo de cambio.	feat: agrega validación de 2FN
Versionado	Aplicar MAJOR.MINOR.PATCH para entregables publicables.	1.0.0, 1.1.0
Revisión	Revisar código antes de fusionar cambios importantes.	Pull request o checklist manual
Seguridad	No subir .env ni claves de Supabase.	.gitignore obligatorio
Automatización	Ejecutar pruebas en push o pull request cuando exista CI.	GitHub Actions tests.yml

Prefijo	Uso	Ejemplo de commit
feat	Nueva funcionalidad.	feat: agrega endpoint de normalización
fix	Corrección de error.	fix: corrige parser de claves primarias
docs	Documentación.	docs: actualiza guía de integración API
test	Pruebas.	test: agrega pruebas para SQL inválido
refactor	Mejora interna sin cambiar comportamiento.	refactor: separa lógica de SupabaseRepository
chore	Tareas de mantenimiento.	chore: actualiza dependencias

4. PROCEDIMIENTOS Y FUNCIONES DEFINIDOS POR EL USUARIO

DataQuest no depende necesariamente de procedimientos almacenados SQL para ejecutar el flujo de negocio. El estándar define que las operaciones críticas se encapsulan en funciones y métodos Python equivalentes a procedimientos de negocio. Esta decisión facilita pruebas unitarias, reutilización desde API, interfaz Streamlit y extensión de VS Code.

4.1 Métodos equivalentes a procedimientos almacenados

Módulo / acción	Método equivalente	Parámetros de entrada	Propósito principal	Estado
-----------------	--------------------	-----------------------	---------------------	--------

Análisis de normalización	procesar_esquema	input_data, formato, fds_input	Ejecuta parsing, diagnóstico y preparación de resultado.	Implementado en controlador
API de normalización	normalizar_esquema	RequestNormalizacion(sql)	Expone diagnóstico mediante POST /api/normalizacion.	Implementado en api.py
Parsing SQL	parse_sql	sql_text	Extrae tablas, columnas, PK y FK desde CREATE TABLE.	Implementado en core/parser.py
Parsing general	parse_schema	input_data, formato, fds_input	Adapta SQL, CSV, XLS o texto a estructura de esquema.	Implementado en core/parser.py
Dependencias funcionales	parse_fds	fds_input	Convierte texto de dependencias A,B -> C en lista estructurada.	Implementado en core/parser.py
Diagnóstico	diagnosticar_esquema	esquema_parseado	Consolida violaciones de normalización por tabla y nivel.	Implementado en core/diagnostico.py
Validación 1FN	validar_1fn	tabla	Detecta incumplimientos de atomicidad y grupos repetidos.	Implementado en core/validador_1fn.py
Validación 2FN	validar_2fn	tabla, fds	Detecta dependencias parciales.	Implementado en core/validador_2fn.py
Validación 3FN	validar_3fn	tabla, fds	Detecta dependencias transitivas.	Implementado en core/validador_3fn.py
Sugerencias	generar_sugerencias	diagnostico	Genera recomendaciones de corrección.	Implementado en core/corrector.py
Historial	guardar_historial	user_id, nombre_esquema, schema_parseado, diagnostico	Registra evidencia en historial_validaciones.	Implementado en controlador / validar integración
Presencia	update_current_user_activity	sin parámetro o sesión actual	Actualiza actividad del usuario conectado.	Implementado en comunidad_controller.py

4.2 Funciones puras

Las funciones puras o de bajo acoplamiento deben calcular, validar o transformar datos sin modificar estado externo. Son ideales para probar la lógica central de normalización.

Función	Parámetros	Retorno	Regla	Ejemplo de uso
calcular_clausura	atributos, dependencias_funcionales	set/list de atributos	No modifica base de datos.	Determinar atributos alcanzables por dependencias.
obtener_claves_candidatas	atributos, dependencias_funcionales	list	Debe ser determinística.	Analizar posibles claves candidatas.
inferir_dependencias_heuristica	tabla	list[dict]	Puede marcarse como heurística.	Sugerir dependencias a revisar.
validar_1fn	tabla	list/dict	Debe retornar violaciones claras.	Detectar valores multivaluados.
validar_2fn	tabla, fds	list/dict	Depende de FDs ingresadas o inferidas.	Detectar dependencia parcial.
validar_3fn	tabla, fds	list/dict	No debe persistir resultados.	Detectar dependencia transitiva.
generar_sugerencias	diagnostico	list[dict]	Debe producir sugerencias accionables.	Separar tabla o normalizar atributos.
aplicar_mejoras	esquema_original, sugerencias_seleccionadas	dict/sql	Debe conservar trazabilidad.	Generar esquema final sugerido.

4.3 Funciones de tabla o procesamiento de colecciones

Función	Entrada	Salida esperada	Propósito
obtener_tablas_del_esquema	esquema_parseado	list[dict]	Listar tablas detectadas para validación.
obtener_campos_por_tabla	tabla	list[dict]	Separar campos, tipos y restricciones.
filtrar_violaciones_por_nivel	violaciones, nivel	list[dict]	Mostrar errores por 1FN, 2FN o 3FN.
listar_dependencias_por_tabla	dependencias, tabla	list[dict]	Agrupar dependencias por entidad.
listar_sugerencias_por_prioridad	sugerencias	list[dict]	Ordenar recomendaciones para el usuario.
get_dashboard_metrics	user_id	dict	Obtener indicadores del historial de usuario.
get_global_metrics	ninguno	dict	Obtener métricas globales para dashboard.

5. ESTÁNDARES DE BASE DE DATOS

La base de datos de DataQuest se encuentra orientada a PostgreSQL/Supabase. El diseño debe mantener nombres descriptivos en snake_case, claves primarias UUID, relaciones explícitas, restricciones de dominio y campos JSONB solo cuando la estructura interna sea flexible.

Regla	Aplicación en DataQuest	Justificación
snake_case	usuarios, perfiles, presencia, historial_validaciones	Mantiene consistencia con Python y SQL.
UUID	id, user_id	Evita colisiones y facilita integración con Supabase.
timestamp with time zone	creado_en, conectado_en, ultima_actividad, fecha	Permite trazabilidad temporal.
text + CHECK	formato_entrada, nivel_inicial, nivel_objetivo, nivel_final	Controla valores permitidos sin crear ENUM rígido.
jsonb	esquema_original, esquema_final, dependencias, violaciones, sugerencias	Permite almacenar resultados dinámicos del motor.
UNIQUE	usuarios.username, presencia.user_id	Evita duplicidad de usuario y presencia.
FK	perfiles.id, presencia.user_id, historial_validaciones.user_id	Mantiene integridad referencial con usuarios.
No contraseña plana	password_hash	Protege credenciales.
Índices recomendados	historial_validaciones.user_id, fecha, nivel_final	Mejora consultas de historial y dashboard.

5.1 Tablas reales consideradas por el estándar

Tabla	Campos principales	Propósito	Reglas críticas
usuarios	id, username, password_hash, creado_en	Gestionar credenciales básicas.	username único; password_hash obligatorio.
perfiles	id, nombre, avatar_url	Separar datos de perfil de autenticación.	id referencia usuarios.id.
presencia	id, user_id, nombre, conectado_en, ultima_actividad	Registrar conexión y actividad.	user_id único y FK a usuarios.id.
historial_validaciones	id, user_id, fecha, nombre_esquema, formato_entrada, nivel_inicial, nivel_objetivo, nivel_final, esquema_original, esquema_final, dependencias, violaciones, sugerencias	Guardar evidencia de validación.	CHECK para formatos y niveles; JSONB para resultados.

6. ESTÁNDARES DE API FASTAPI

La API de DataQuest debe exponer servicios HTTP/JSON con rutas claras, modelos de entrada/salida, validaciones, códigos HTTP correctos y documentación OpenAPI. La ruta principal es POST /api/normalizacion.

- Usar prefijo /api para endpoints funcionales.
- Usar Pydantic para request y response.
- Validar entrada antes de invocar el parser.
- Retornar JSON uniforme con success, data o error.
- Usar códigos HTTP correctos: 200, 400, 404, 422, 500.
- No exponer stack traces ni rutas internas en respuestas públicas.
- Configurar CORS de manera restrictiva en producción.
- Documentar endpoints mediante Swagger/OpenAPI.

Endpoint	Método	Entrada esperada	Respuesta estándar	Regla
/	GET	Ninguna	status, message, docs_url	Solo para healthcheck básico.
/api/normalizacion	POST	sql	success, data/error, diagnóstico	Debe analizar SQL como texto, nunca ejecutarlo.
/docs	GET	Ninguna	Documentación Swagger	Uso interno o académico.

Contrato recomendado para POST /api/normalizacion:

Campo	Tipo	Obligatorio	Descripción
sql	str	Sí	Sentencias CREATE TABLE que definen el esquema relacional.
nombre_esquema	str	Recomendado	Nombre del esquema para historial.
formato_entrada	str	Opcional	Formato de entrada; por defecto sql o texto_pegado.
nivel_objetivo	str	Opcional	Nivel buscado: 1fn, 2fn o 3fn.
dependencias	list[dict]	Opcional	Dependencias funcionales suministradas.
user_id	uuid	Opcional	Usuario asociado si hay sesión autenticada.

7. ESTÁNDARES DE INTERFAZ STREAMLIT

La interfaz Streamlit debe actuar como capa de presentación. No debe contener lógica compleja del motor, ni consultas directas si ya existe un controlador o repositorio. Debe presentar entradas, botones, resultados, historial y mensajes de forma clara para el usuario académico.

- Separar carga de archivo, ingreso de SQL, análisis y visualización de resultados.
- Validar entradas antes de invocar el motor o la API.
- Usar st.session_state solo para datos de sesión necesarios.
- No exponer claves de Supabase ni variables de entorno en pantalla.
- Mostrar errores de forma amigable y evitar stack traces.
- Permitir limpiar entradas y resultados.
- Presentar violaciones y sugerencias en tablas claras.
- Mantener coherencia visual entre páginas de inicio, validador, resultados e historial.

Componente	Función	Regla de implementación
Área SQL	Permite pegar sentencias CREATE TABLE.	Limitar tamaño y validar que no esté vacía.
Selector de formato	Define sql, csv, xls, txt o texto_pegado.	Debe coincidir con CHECK de la base de datos.
Selector de nivel	Define nivel objetivo de normalización.	Aceptar solo 1fn, 2fn o 3fn.
Botón validar	Ejecuta el análisis.	Debe bloquear entradas inválidas.
Panel de resultados	Muestra nivel final y cumplimiento.	No mezclar diagnóstico con edición manual.
Tabla de violaciones	Lista incumplimientos.	Agrupar por nivel y tabla.
Tabla de sugerencias	Lista recomendaciones.	Ordenar por prioridad o nivel.
Historial	Muestra validaciones previas.	Consultar por user_id y limitar resultados.

8. ESTÁNDARES DE SEGURIDAD

DataQuest es un sistema académico, pero debe cumplir reglas mínimas de seguridad para proteger credenciales, datos de usuario, variables de entorno y disponibilidad del sistema. El SQL ingresado por el usuario se considera texto de análisis; no debe ejecutarse contra una base de datos productiva.

- El campo password_hash es obligatorio y nunca debe reemplazarse por contraseña plana.
- No imprimir contraseñas, tokens, claves de Supabase ni variables de entorno en consola o UI.
- No subir .env al repositorio.
- Validar SQL recibido para evitar ejecución accidental.
- Limitar tamaño máximo de entrada SQL.
- Sanitizar mensajes de error antes de mostrarlos al usuario.
- Proteger endpoints sensibles si se agrega autenticación.
- Restringir CORS en producción.
- Separar claves públicas y claves de servicio de Supabase.
- Mantener backups o exportaciones de esquema cuando el proyecto avance.

Riesgo	Regla de mitigación	Responsable
Credenciales expuestas	Usar .env y .gitignore; no imprimir tokens.	Desarrollador
SQL ejecutado por error	Procesar SQL solo como texto; no usar conexión para ejecutar entrada del usuario.	Motor core

Errores técnicos visibles	Mostrar mensajes controlados y registrar detalle internamente.	API / UI
Datos inconsistentes	Usar Pydantic + CHECK + validaciones de dominio.	API / BD
CORS abierto en producción	Limitar dominios permitidos al desplegar.	DevOps
Historial no trazable	Guardar fecha, user_id y nombre_esquema.	Persistencia

9. ESTÁNDARES DE PRUEBAS Y QA

Las pruebas deben verificar que el sistema maneje entradas correctas, errores esperados, persistencia y respuestas de API. El objetivo no es probar únicamente que el sistema funcione en un caso ideal, sino asegurar que falle de manera controlada cuando la entrada es incompleta o inválida.

ID	Caso	Entrada	Resultado esperado	Tipo de prueba	Estado
QA-01	SQL vacío	""	Error controlado: el SQL no puede estar vacío.	Unit/API	Obligatorio
QA-02	SQL inválido	CREATE usuario id int	Mensaje de sintaxis o tablas no detectadas.	Unit	Obligatorio
QA-03	CREATE TABLE simple válido	CREATE TABLE alumno (...)	Esquema parseado con tabla y campos.	Unit	Obligatorio
QA-04	Tabla sin clave primaria	CREATE TABLE sin PK	Violación o advertencia de diseño.	Unit	Obligatorio
QA-05	Atributos multivaluados	campo telefonos o lista	Violación relacionada con 1FN.	Unit	Obligatorio
QA-06	Dependencia parcial	PK compuesta + FD parcial	Violación de 2FN.	Unit	Obligatorio
QA-07	Dependencia transitiva	A -> B, B -> C	Violación de 3FN.	Unit	Obligatorio
QA-08	Formato inválido	formato_entrada=pdf	Rechazo por dominio permitido.	API/BD	Obligatorio
QA-09	Nivel objetivo inválido	nivel_objetivo=4fn	Error 400/422 o CHECK.	API/BD	Obligatorio
QA-10	Guardar historial	resultado válido	Registro en historial_validaciones.	Integration	Obligatorio
QA-11	Consulta de historial	user_id válido	Lista de validaciones del usuario.	Integration	Obligatorio
QA-12	Endpoint normalización	POST /api/normalizacion	Respuesta 200 con success/data o error controlado.	API	Obligatorio
QA-13	Entrada grande	SQL muy extenso	Respuesta controlada sin bloquear UI.	Performance	Recomendado
QA-14	UI sin sesión	Acceso a vista protegida	Redirección o mensaje de autenticación.	UI	Recomendado

9.1 Criterios de aceptación QA

- El proyecto debe ejecutar pruebas unitarias del core antes de la entrega.
- El endpoint /api/normalizacion debe validar entradas vacías o mal formadas.
- Los resultados deben contener estructura JSON uniforme.
- El historial debe guardar nombre_esquema, niveles, violaciones y sugerencias cuando corresponda.
- La interfaz debe mostrar errores sin romper la ejecución completa.
- No deben existir credenciales reales en el repositorio ni en capturas de evidencia.

10. BENEFICIOS

- Aumenta la mantenibilidad del proyecto al establecer nombres, capas y responsabilidades claras.
- Reduce duplicidad entre API, Streamlit y motor core al promover funciones reutilizables.
- Mejora la trazabilidad del análisis mediante fechas, historial y estructuras JSONB documentadas.
- Refuerza la seguridad al prohibir contraseñas planas, credenciales expuestas y ejecución de SQL del usuario.
- Facilita la sustentación académica porque cada módulo tiene propósito, reglas y ejemplos.
- Permite extender DataQuest con reportes, extensión VS Code o más validadores sin romper la arquitectura.
- Mejora el control de errores y la experiencia del usuario al estandarizar respuestas y mensajes.
- Favorece la automatización de pruebas y la integración continua mediante convenciones de Git y QA.
- Alinea Python, FastAPI, Streamlit y Supabase/PostgreSQL bajo un mismo criterio técnico.
- Ayuda a futuros integrantes a comprender el proyecto sin depender del autor original.

11. CONCLUSIONES

- El estándar de programación propuesto permite que DataQuest mantenga una estructura clara, modular y escalable, adecuada para un sistema académico de validación de normalización.
- La aplicación de PEP 8, nombres descriptivos, funciones con responsabilidad única y separación por capas contribuye directamente a la calidad del código y al mantenimiento del proyecto.
- La integración entre modelos Pydantic, validaciones Python y restricciones CHECK en PostgreSQL/Supabase fortalece la consistencia de los datos y reduce errores de entrada.
- El uso de JSONB en `historial_validaciones` permite almacenar resultados flexibles del motor de normalización sin sobredimensionar el modelo físico de la base de datos.
- La separación entre interfaz Streamlit, API FastAPI, motor core y repositorios de datos reduce el acoplamiento y facilita pruebas unitarias, de integración y de interfaz.
- La definición de estándares de seguridad y QA convierte a DataQuest en un proyecto más defendible, confiable y preparado para futuras mejoras académicas.

12. RECOMENDACIONES

- Mantener este documento actualizado cada vez que se agreguen módulos, endpoints, tablas o validadores.
- Aplicar revisión de código antes de cada entrega académica importante.
- Crear pruebas unitarias para cada función crítica del motor core: parser, 1FN, 2FN, 3FN, diagnóstico y sugerencias.
- No mezclar lógica de interfaz con lógica de normalización; la UI debe delegar en controladores o servicios.
- Centralizar el acceso a Supabase/PostgreSQL en una capa de conexión o repositorio.
- Mantener archivos `.env` fuera del repositorio y documentar variables necesarias en `.env.example`.
- Usar herramientas de calidad como black, ruff, flake8, pylint o pytest-cov cuando sea posible.
- Documentar todo endpoint relevante en Swagger/OpenAPI y en `INTEGRACION_API.md`.
- Agregar índices en `historial_validaciones` si el volumen de validaciones aumenta.
- Registrar evidencias de pruebas y capturas limpias para la sustentación.